

ClickHouse: A Comprehensive Technical Guide

Authors: Julius Ball, Anton Pfurtscheller

Table of Contents

1. Introduction to ClickHouse
 2. Column-Oriented vs Row-Oriented Storage
 3. Architecture Deep Dive
 4. The MergeTree Engine Family
 5. Performance Characteristics and Benchmarks
 6. Core Features and Capabilities
 7. SQL Dialect and Query Examples
 8. Real-World Use Cases
 9. When NOT to Use ClickHouse
 10. Production Deployments and Case Studies
 11. Installation, Setup, and Hosting Options
 12. Best Practices and Optimization
 13. Comparison with Other OLAP Systems
 14. Resources and Further Reading
-

1. Introduction to ClickHouse

1.1 What is ClickHouse?

ClickHouse is an open-source, column-oriented database management system (DBMS) specifically designed for Online Analytical Processing (OLAP) workloads. Originally developed by Yandex, the Russian technology company, ClickHouse was created to power Yandex.Metrica—one of the world’s largest web analytics platforms, processing over 13 trillion records and more than 20 billion events daily.

The project was released as open-source software in June 2016 under the Apache 2.0 license, and has since gained widespread adoption across industries ranging from ad-tech and e-commerce to financial services and IoT platforms.

1.2 Core Design Principles

ClickHouse was built around several fundamental principles that distinguish it from traditional database systems:

Speed Above All Else: ClickHouse prioritizes query execution speed, often at the expense of other features. The system is designed to process analytical queries over massive datasets in milliseconds to seconds, rather than minutes or hours.

Columnar Storage: Data is stored by columns rather than rows, enabling efficient compression and allowing queries to read only the columns they need—dramatically reducing I/O for analytical workloads.

Vectorized Query Execution: ClickHouse processes data in batches (vectors) using CPU SIMD instructions, maximizing hardware utilization and achieving throughput measured in billions of rows per second.

Real-Time Data Ingestion: Unlike many analytical databases that require batch loading, ClickHouse supports real-time data insertion with sub-second latency to query availability.

SQL Compatibility: ClickHouse uses a SQL-compatible query language with extensions for analytical functions, making it accessible to developers and analysts familiar with standard SQL.

1.3 Historical Context

The development of ClickHouse began in 2008-2009 at Yandex, driven by the need to process web analytics data at unprecedented scale. The existing solutions at the time—including MySQL, PostgreSQL, and commercial analytical databases—could not meet the performance requirements for interactive dashboards over billions of events.

Key milestones in ClickHouse’s evolution:

- **2008-2009:** Initial development begins at Yandex
- **2012:** Production deployment for Yandex.Metrica
- **2016:** Open-source release under Apache 2.0
- **2021:** ClickHouse Inc. founded as an independent company
- **2022:** ClickHouse Cloud launched as a managed service
- **2023-Present:** Continued growth with enterprise features and cloud-native capabilities

1.4 Key Differentiators

What sets ClickHouse apart from other analytical databases:

Feature	ClickHouse Approach
Storage Format	True columnar with per-column compression
Query Engine	Vectorized execution with SIMD optimization
Scalability	Both vertical and horizontal
Dependencies	Zero external dependencies (self-contained)

Feature	ClickHouse Approach
Deployment	Single binary, runs anywhere
Cost	Open-source with commercial support options

2. Column-Oriented vs Row-Oriented Storage

2.1 Understanding Row-Oriented Storage

Traditional relational databases like PostgreSQL, MySQL, Oracle, and SQL Server use row-oriented storage. In this model, all values for a single row are stored contiguously on disk:

Row Storage Layout:

```
[Row1: id=1, name="John", email="john@example.com", age=35, gender="M"]
```

```
[Row2: id=2, name="Jane", email="jane@example.com", age=29, gender="F"]
```

```
[Row3: id=3, name="Bill", email="bill@example.com", age=42, gender="M"]
```

Advantages of Row Storage:

- Efficient for transactional workloads (INSERT, UPDATE, DELETE)
- Fast retrieval of entire rows by primary key
- Natural fit for OLTP applications
- Simple implementation of ACID transactions

Disadvantages for Analytics:

- Must read entire rows even when only needing a few columns
- Poor compression ratios due to mixed data types
- Inefficient for aggregations over large datasets

2.2 Column-Oriented Storage in ClickHouse

ClickHouse stores data by columns, meaning all values for a single column are stored together:

Column Storage Layout:

```
[id column]:      [1, 2, 3, 4, 5, ...]
[name column]:    ["John", "Jane", "Bill", ...]
[email column]:   ["john@...", "jane@...", "bill@...", ...]
[age column]:     [35, 29, 42, ...]
[gender column]:  ["M", "F", "M", ...]
```

2.3 Why Columnar Storage Excels for Analytics

Reduced I/O: Consider a query like `SELECT AVG(age) FROM users`. In a row-oriented database, this query must read every column of every row, even though it only needs the `age` column. With columnar storage, only the `age` column is read from disk—potentially reducing I/O by 90% or more for wide tables.

Superior Compression: Columnar storage enables dramatically better compression because:

1. **Data Type Homogeneity:** All values in a column share the same data type, enabling type-specific compression algorithms
2. **Value Locality:** Similar values are stored together, improving compression ratios
3. **Pattern Recognition:** Repeated values, sequences, and patterns are easier to detect and compress

Typical compression ratios in ClickHouse:

Data Type	Typical Compression Ratio
Integers (sorted)	20-100x
Timestamps (sorted)	10-50x
Low-cardinality strings	50-200x
High-cardinality strings	3-10x
Floating point	2-5x

Real-World Impact: ClickHouse commonly achieves 10-20x compression compared to raw data, and uses approximately 6x less storage than PostgreSQL for equivalent datasets.

2.4 Vectorized Query Execution

Beyond storage format, ClickHouse’s query execution model is fundamentally different from row-oriented databases. Instead of processing one row at a time, ClickHouse processes data in vectors (batches) of rows.

Row-at-a-time Processing (Traditional):

```
for each row in table:
    if row.status == 'active':
        sum += row.amount
```

Vectorized Processing (ClickHouse):

```
load 8192 status values into vector
load 8192 amount values into vector
compare status vector against 'active' → bitmap
sum amount vector where bitmap is set (using SIMD)
```

2.5 CPU SIMD Instructions

SIMD (Single Instruction, Multiple Data) instructions allow modern CPUs to perform the same operation on multiple data elements simultaneously. ClickHouse heavily leverages SIMD for:

- Arithmetic operations (sum, multiply, etc.)
- Comparisons and filtering
- String operations
- Compression/decompression
- Hash calculations

SIMD Register Widths:

Instruction Set	Register Width	Elements per Operation (64-bit)
SSE4.2	128 bits	2
AVX2	256 bits	4
AVX-512	512 bits	8

With AVX-512, ClickHouse can process 8 Int64 values in a single CPU cycle. Combined with multiple CPU cores and vectorized execution, this enables scan rates of billions of rows per second.

2.6 Performance Comparison

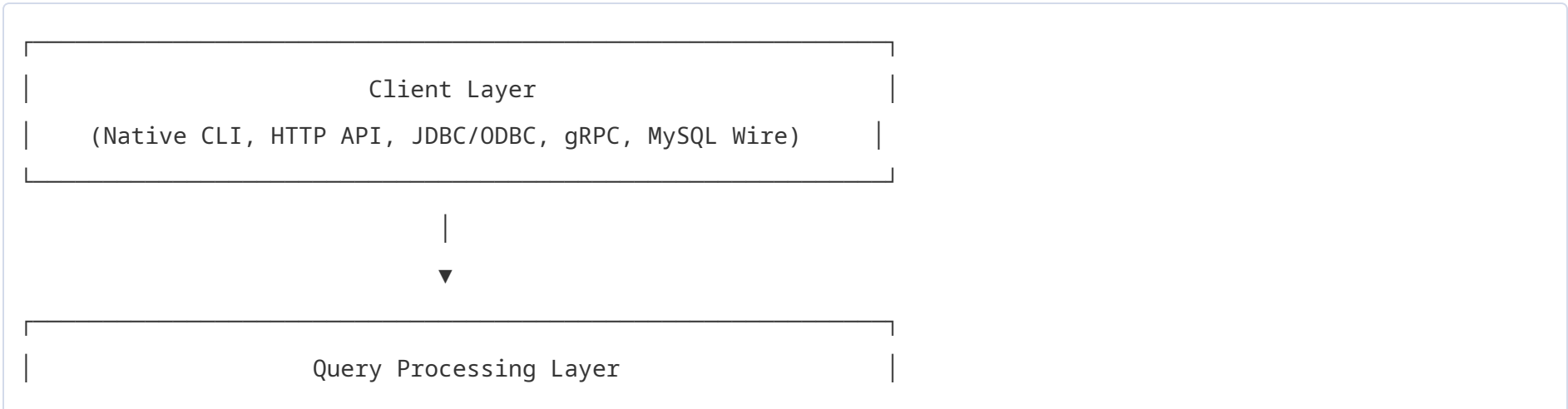
The combination of columnar storage, compression, and vectorized execution yields dramatic performance improvements for analytical queries:

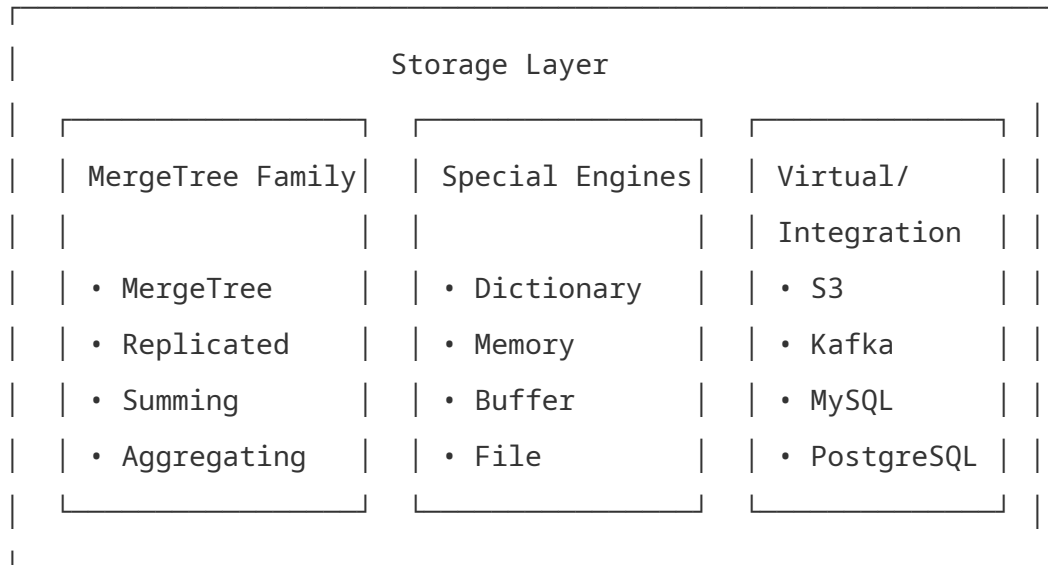
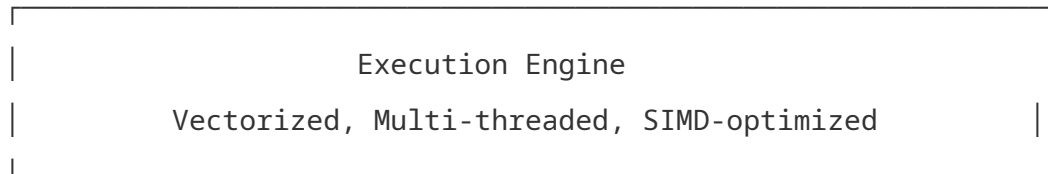
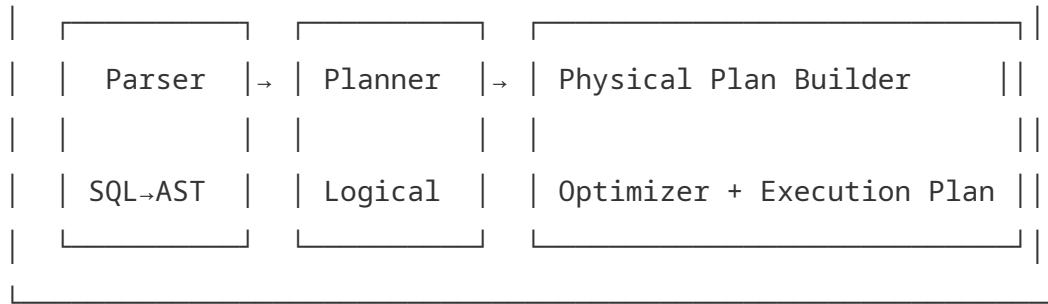
Metric	PostgreSQL	ClickHouse	Improvement
Storage Size	100 GB	15-20 GB	5-7x smaller
Simple Aggregation	30 seconds	0.3 seconds	100x faster
Complex Analytics	5 minutes	3 seconds	100x faster
Scan Rate	1-10M rows/sec	1-10B rows/sec	100-1000x

3. Architecture Deep Dive

3.1 High-Level Architecture

ClickHouse’s architecture is remarkably clean and modular, consisting of several distinct layers:





3.2 Zero External Dependencies

One of ClickHouse's most distinctive architectural features is its complete lack of external dependencies. Unlike systems that require:

- Separate coordination services (ZooKeeper, etcd)
- External storage systems (HDFS, S3)
- Message queues (Kafka)
- Caching layers (Redis, Memcached)

ClickHouse can run as a completely self-contained single binary. This dramatically simplifies deployment and operations:

- **Single Node Deployment:** Download, configure, run
- **No Coordination Overhead:** No ZooKeeper latency or availability concerns
- **Simplified Debugging:** All components in one process
- **Reduced Operational Complexity:** Fewer moving parts to monitor and maintain

For distributed deployments, ClickHouse includes its own coordination system called ClickHouse Keeper, which is compatible with the ZooKeeper protocol but runs embedded within ClickHouse.

3.3 Query Processing Pipeline

Step 1: Parsing

The parser transforms SQL text into an Abstract Syntax Tree (AST). ClickHouse's parser supports:

- Standard SQL syntax
- ClickHouse-specific extensions

- Multiple SQL dialects for compatibility

Step 2: Query Planning

The planner analyzes the AST and creates a logical query plan. This stage:

- Resolves table and column references
- Validates data types and function signatures
- Applies semantic analysis

Step 3: Optimization

The optimizer transforms the logical plan into an efficient physical plan:

- Predicate pushdown (push filters as close to storage as possible)
- Column pruning (read only required columns)
- Partition pruning (skip irrelevant data partitions)
- Index utilization (use primary key and skip indexes)
- Join optimization (choose optimal join algorithms)
- Aggregation optimization (use pre-aggregated data when available)

Step 4: Execution

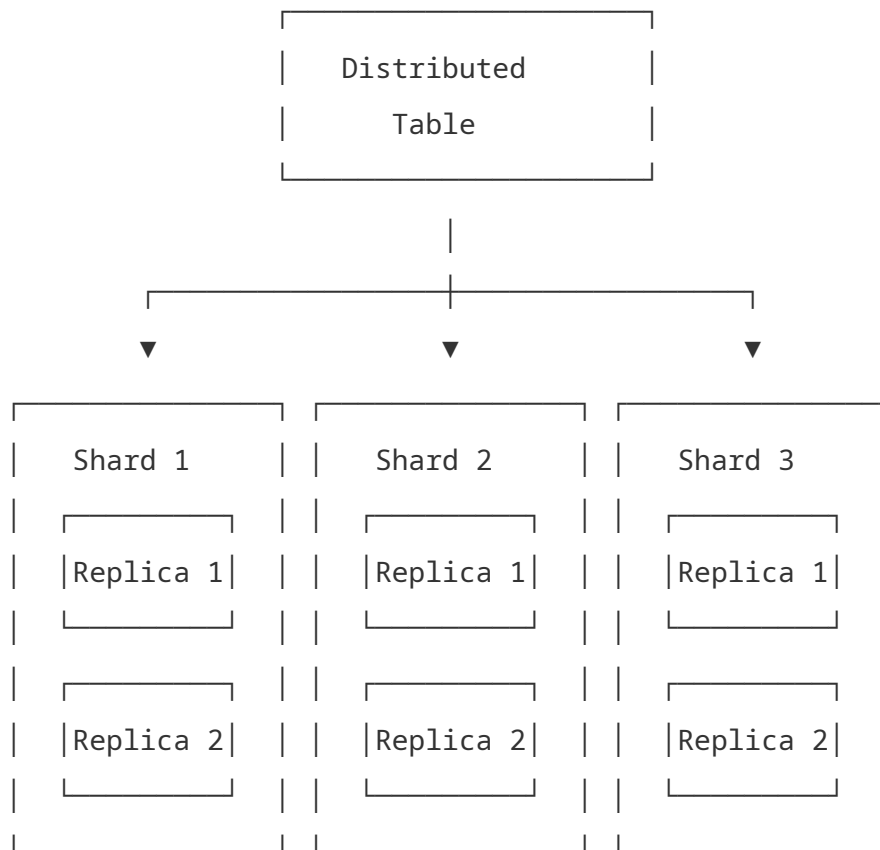
The execution engine runs the physical plan:

- Parallel execution across CPU cores
- Vectorized processing of data blocks

- Streaming results to reduce memory usage
- Distributed execution across cluster nodes

3.4 Distributed Architecture

ClickHouse supports horizontal scaling through sharding and replication:



Sharding: Data is partitioned across multiple nodes (shards) to distribute storage and processing load. Each shard contains a subset of the total data.

Replication: Each shard can have multiple replicas for high availability and read scaling. Replicas are kept in sync through ClickHouse’s built-in replication protocol.

Distributed Queries: When querying a distributed table, ClickHouse:

- 1. Sends sub-queries to all relevant shards
- 2. Collects partial results from each shard
- 3. Merges results into a final response

3.5 Access Layer Protocols

ClickHouse supports multiple protocols for client connectivity:

Protocol	Port	Use Case
Native TCP	9000	High-performance CLI and drivers
HTTP	8123	REST API, web interfaces
MySQL Wire	9004	MySQL client compatibility
PostgreSQL Wire	9005	PostgreSQL client compatibility
gRPC	9100	Modern RPC-based clients

This flexibility allows ClickHouse to integrate with virtually any application or BI tool.

4. The MergeTree Engine Family

4.1 Understanding Table Engines

ClickHouse uses a pluggable storage engine architecture. Each table is associated with an engine that determines:

- How data is stored on disk
- How data is indexed
- Whether replication is supported
- How concurrent access is handled
- What optimizations are available

The MergeTree family is the primary engine family for production workloads, but ClickHouse includes over 30 different engines for various purposes.

4.2 The Base MergeTree Engine

MergeTree is the foundation of ClickHouse's storage system. It's designed for:

- Inserting large volumes of data
- Fast analytical queries
- Automatic background optimization

Creating a MergeTree Table:

```
CREATE TABLE events (  
    timestamp DateTime,  
    user_id UInt64,  
    event_type String,  
    value Float64
```

```
) ENGINE = MergeTree()  
ORDER BY (timestamp, user_id);
```

Key Concepts:

Parts: Data is stored in immutable “parts” (directories containing column files). Each INSERT creates a new part. Parts are never modified in place.

Merges: Background processes continuously merge smaller parts into larger ones, optimizing storage and query performance. This is where “MergeTree” gets its name.

Primary Key/ORDER BY: The ORDER BY clause defines how data is sorted within each part. This sorting:

- Enables efficient range queries
- Improves compression ratios
- Creates a sparse primary index

Sparse Primary Index: Unlike traditional B-tree indexes that index every row, MergeTree creates an index entry every 8192 rows (configurable). This keeps the index small enough to fit in memory while still enabling efficient data location.

4.3 MergeTree Family Variants

ReplacingMergeTree - For Deduplication

Automatically removes duplicate rows based on the ORDER BY key during merges:

```
CREATE TABLE user_profiles (  
    user_id UInt64,  
    name String,  
    email String,  
    updated_at DateTime  
) ENGINE = ReplacingMergeTree(updated_at)  
ORDER BY user_id;
```

The **updated_at** column determines which row to keep when duplicates exist. Note: Deduplication is eventual (happens during merges), not immediate.

SummingMergeTree - For Automatic Aggregation

Automatically sums numeric columns for rows with the same ORDER BY key:

```
CREATE TABLE daily_stats (  
    date Date,  
    category String,  
    views UInt64,  
    clicks UInt64,  
    revenue Float64  
) ENGINE = SummingMergeTree()  
ORDER BY (date, category);
```

When rows with the same (date, category) are merged, views, clicks, and revenue are summed automatically.

AggregatingMergeTree - For Complex Pre-Aggregation

Stores intermediate aggregation states that can be combined later:

```
CREATE TABLE hourly_uniques (  
    hour DateTime,  
    page String,  
    unique_users AggregateFunction(uniq, UInt64)  
) ENGINE = AggregatingMergeTree()  
ORDER BY (hour, page);
```

CollapsingMergeTree - For Mutable Data

Supports “collapsing” rows with matching keys but opposite signs, enabling efficient updates and deletes:

```
CREATE TABLE user_balances (  
    user_id UInt64,  
    balance Float64,  
    sign Int8 -- 1 for insert, -1 for delete  
) ENGINE = CollapsingMergeTree(sign)  
ORDER BY user_id;
```

VersionedCollapsingMergeTree - Collapsing with Version Control

Adds version tracking to collapsing behavior for more complex update scenarios.

GraphiteMergeTree - For Graphite Metrics

Optimized for storing and querying Graphite-compatible time series data.

4.4 Complete MergeTree Variant Summary

Engine	Primary Use Case	Key Feature
MergeTree	General OLAP	Foundation engine
ReplacingMergeTree	Deduplication	Keeps latest version
SummingMergeTree	Pre-aggregation	Auto-sums numeric columns
AggregatingMergeTree	Complex aggregates	Stores aggregate states
CollapsingMergeTree	Mutable data	Sign-based row collapsing
VersionedCollapsingMergeTree	Versioned mutations	Version + sign collapsing
GraphiteMergeTree	Graphite metrics	Rollup and retention
ReplicatedMergeTree*	High availability	Multi-node replication

*Note: Any MergeTree variant can be prefixed with “Replicated” (e.g., ReplicatedSummingMergeTree) for distributed replication.

4.5 Other Engine Families

Log Family - For Temporary/Staging Data

Simple append-only engines without indexes or background merges:

- **Log** : Multi-threaded reads
- **TinyLog** : Single-threaded, minimal overhead
- **StripeLog** : Optimized for small tables with many columns

Integration Engines - For External Data Sources

- **MySQL** : Query MySQL tables directly
- **PostgreSQL** : Query PostgreSQL tables
- **MongoDB** : Query MongoDB collections
- **S3** : Query files in S3 buckets
- **Kafka** : Stream data from Kafka topics
- **HDFS** : Access Hadoop filesystem
- **JDBC** : Generic JDBC connectivity

Special Purpose Engines

- **Memory** : In-memory storage (lost on restart)
- **Buffer** : Write buffer for batching inserts
- **Null** : Discards all data (useful for testing)
- **File** : Read/write local files (CSV, JSON, Parquet)
- **URL** : Query data from HTTP endpoints
- **Dictionary** : Key-value lookups with caching
- **Distributed** : Logical view across sharded tables
- **MaterializedView** : Continuously updated aggregations

5. Performance Characteristics and Benchmarks

5.1 Performance at Scale

ClickHouse’s performance characteristics make it exceptional for large-scale analytical workloads:

Scan Rates: ClickHouse can process billions of rows per second on modern hardware. A single server can often achieve:

- 1-2 billion rows/second for simple aggregations
- 100-500 million rows/second for complex queries
- 50-200 GB/second of compressed data throughput

Query Latency: For typical analytical queries:

- Simple aggregations: 10-100 milliseconds
- Complex multi-table queries: 100ms - 2 seconds
- Full table scans on TB-scale data: 1-10 seconds

5.2 Benchmark Comparisons

ClickHouse vs. Traditional RDBMS (PostgreSQL)

In the benchmark referenced from the ClickHouse blog (JSON documents with 1 billion records):

System	Cold Query	Hot Query
ClickHouse	11.85 sec	5.63 sec
MongoDB	5.99 hours	5.94 hours
Elasticsearch	51.49 sec	45.51 sec

System	Cold Query	Hot Query
DuckDB	1.04 hours	1.04 hours
PostgreSQL	9.06 hours	9.06 hours

ClickHouse completed the aggregation query approximately **5,800x faster** than PostgreSQL and **3,800x faster** than MongoDB.

ClickBench Results

ClickBench (clickhouse.com/benchmark) compares analytical databases on a standardized workload. ClickHouse consistently ranks among the fastest:

Metric	ClickHouse	Closest Competitor
Total Query Time	1x (baseline)	2-10x slower
Storage Size	Smallest	2-5x larger
Load Time	Fastest	2-3x slower

5.3 Factors Affecting Performance

Hardware Utilization

ClickHouse is designed to maximize hardware utilization:

- **CPU:** Vectorized execution saturates all cores
- **Memory:** Efficient caching and streaming processing
- **Disk:** Parallel I/O with asynchronous reads

- **Network:** Compressed data transfer in distributed queries

Query Performance Factors

1. **Primary Key Design:** Queries filtering on ORDER BY columns are dramatically faster
2. **Partition Pruning:** Date-based partitioning enables skipping irrelevant data
3. **Column Selection:** Selecting fewer columns reduces I/O
4. **Compression:** Better compression means less data to read
5. **Data Locality:** Hot data in page cache yields sub-millisecond responses

5.4 Linear Scalability

ClickHouse demonstrates near-linear scalability in both directions:

Vertical Scaling: Adding CPU cores or faster storage yields proportional improvements. A 64-core server processes queries roughly 8x faster than an 8-core server.

Horizontal Scaling: Adding shards distributes both storage and processing. Query performance scales linearly with shard count for scan-heavy workloads.

6. Core Features and Capabilities

6.1 Real-Time Data Ingestion

Unlike batch-oriented data warehouses, ClickHouse supports continuous, real-time data ingestion:

High Insert Throughput: A single ClickHouse server can ingest:

- 100,000+ rows per second via individual INSERTs

- 1,000,000+ rows per second via batch INSERTs
- Multi-million rows per second via parallel streams

Immediate Query Availability: Inserted data is queryable within milliseconds, not hours.

Best Practices for Ingestion:

- Batch inserts (1000+ rows per INSERT)
- Use asynchronous inserts for high-frequency streams
- Consider Buffer tables for write smoothing
- Use Kafka engine for streaming pipelines

6.2 Approximate Query Processing

For extremely large datasets where exact results aren't required, ClickHouse provides approximate algorithms:

HyperLogLog for Cardinality Estimation:

```
-- Approximate count of unique users (fast, ~2% error)
SELECT uniqHLL12(user_id) FROM events;

-- Exact count (slower for large datasets)
SELECT uniq(user_id) FROM events;
```

Quantile Estimation:

```
-- Approximate median and percentiles
SELECT
    quantile(0.5)(response_time) as median,
    quantile(0.95)(response_time) as p95,
    quantile(0.99)(response_time) as p99
FROM requests;

-- T-Digest algorithm for better accuracy
SELECT quantileTDigest(0.99)(response_time) FROM requests;
```

Sampling:

```
-- Query 10% sample for faster approximate results
SELECT avg(value) FROM events SAMPLE 0.1;
```

6.3 Materialized Views

Materialized views in ClickHouse are triggers that transform and aggregate data as it's inserted:

```
-- Source table
CREATE TABLE raw_events (
    timestamp DateTime,
    user_id UInt64,
    event_type String,
    value Float64
```



```
) ENGINE = MergeTree()
ORDER BY timestamp;

-- Materialized view for hourly aggregation
CREATE MATERIALIZED VIEW hourly_stats
ENGINE = SummingMergeTree()
ORDER BY (hour, event_type)
AS SELECT
    toStartOfHour(timestamp) as hour,
    event_type,
    count() as event_count,
    sum(value) as total_value,
    uniqState(user_id) as unique_users
FROM raw_events
GROUP BY hour, event_type;
```

Key Benefits:

- Automatic aggregation at insert time
- No separate ETL jobs required
- Query pre-aggregated data instead of raw events
- Can chain multiple materialized views

6.4 Data TTL (Time To Live)

Automatically expire and delete old data:

```
CREATE TABLE logs (  
    timestamp DateTime,  
    level String,  
    message String  
) ENGINE = MergeTree()  
ORDER BY timestamp  
TTL timestamp + INTERVAL 30 DAY; -- Delete after 30 days
```

TTL for Tiered Storage:

```
CREATE TABLE events (  
    timestamp DateTime,  
    data String  
) ENGINE = MergeTree()  
ORDER BY timestamp  
TTL  
    timestamp + INTERVAL 7 DAY TO VOLUME 'hot',  
    timestamp + INTERVAL 30 DAY TO VOLUME 'cold',  
    timestamp + INTERVAL 365 DAY DELETE;
```

6.5 Extensive Function Library

ClickHouse includes over 1000 built-in functions:

Date/Time Functions: `toDate`, `toStartOfMonth`, `dateDiff`, `formatDateTime`, timezone conversions

String Functions: `substring` , `concat` , `lower` , `upper` , regex operations, fuzzy matching

Array Functions: `arrayJoin` , `arrayMap` , `arrayFilter` , `arrayReduce`

Aggregate Functions: `sum` , `avg` , `count` , `min` , `max` , `uniq` , `quantile` , `groupArray` , `topK`

Mathematical Functions: Trigonometry, statistics, random number generation

Encoding Functions: Base64, hex, URL encoding, hashing (MD5, SHA, xxHash)

JSON Functions: `JSONExtract` , `JSONHas` , `JSONLength` , `JSONType`

Geo Functions: Distance calculations, H3 indexing, polygon operations

6.6 Advanced Indexing

Beyond the primary key, ClickHouse supports secondary indexes:

Skip Indexes (Data Skipping):

```
CREATE TABLE events (  
    timestamp DateTime,  
    user_id UInt64,  
    category LowCardinality(String),  
    INDEX idx_category category TYPE set(100) GRANULARITY 4
```

```
) ENGINE = MergeTree()  
ORDER BY timestamp;
```

Index Types:

- **minmax** : Track min/max values per granule
- **set** : Track distinct values (up to N)
- **bloom_filter** : Probabilistic membership test
- **ngrambf_v1** : N-gram bloom filter for substring search
- **tokenbf_v1** : Token bloom filter for full-text search

7. SQL Dialect and Query Examples

7.1 Table Creation

Basic Table with MergeTree:

```
CREATE TABLE events (  
    -- DateTime types  
    timestamp DateTime,  
    event_date Date DEFAULT toDate(timestamp),  
  
    -- Integer types (use smallest sufficient type)  
    user_id UInt64,  
    session_id UInt32,
```

```

-- String types
event_type String,
category LowCardinality(String), -- Optimized for low-cardinality

-- Numeric types
value Float64,
count UInt32 DEFAULT 1,

-- Nullable (use sparingly - adds overhead)
optional_field Nullable(String),

-- Complex types
tags Array(String),
properties Map(String, String)

) ENGINE = MergeTree()
PARTITION BY toYYYYMM(timestamp)
ORDER BY (event_date, user_id, timestamp)
SETTINGS index_granularity = 8192;

```

7.2 Data Insertion

Single Row Insert:

```

INSERT INTO events (timestamp, user_id, event_type, value)

```

```
VALUES ('2024-01-15 10:30:00', 12345, 'page_view', 1.0);
```

Batch Insert:

```
INSERT INTO events (timestamp, user_id, event_type, value)
VALUES
    ('2024-01-15 10:30:00', 12345, 'page_view', 1.0),
    ('2024-01-15 10:30:01', 12346, 'click', 2.5),
    ('2024-01-15 10:30:02', 12347, 'purchase', 99.99);
```

Insert from SELECT:

```
INSERT INTO events_archive
SELECT * FROM events
WHERE event_date < today() - 30;
```

Insert from File:

```
INSERT INTO events FORMAT CSVWithNames
-- Then pipe CSV data or use clickhouse-client with --query
```

7.3 Analytical Queries

Time-Based Aggregation:

```
SELECT
    toDate(timestamp) as day,
    count() as events,
    uniq(user_id) as unique_users,
    sum(value) as total_value
FROM events
WHERE timestamp >= now() - INTERVAL 7 DAY
GROUP BY day
ORDER BY day;
```

Window Functions:

```
SELECT
    user_id,
    timestamp,
    value,
    sum(value) OVER (PARTITION BY user_id ORDER BY timestamp) as running_total,
    row_number() OVER (PARTITION BY user_id ORDER BY timestamp DESC) as recency_rank
FROM events
WHERE event_type = 'purchase';
```

Funnel Analysis:

```
SELECT
    level,
```

```
count() as users
FROM (
  SELECT
    user_id,
    windowFunnel(86400)(
      timestamp,
      event_type = 'visit',
      event_type = 'add_to_cart',
      event_type = 'purchase'
    ) as level
  FROM events
  WHERE timestamp >= today() - 7
  GROUP BY user_id
)
GROUP BY level
ORDER BY level;
```

Retention Analysis:

```
SELECT
  first_day,
  retention_day,
  retained_users,
  round(retained_users / first_day_users * 100, 2) as retention_rate
FROM (
  SELECT
```



```

    first_day,
    dateDiff('day', first_day, activity_day) as retention_day,
    count(DISTINCT user_id) as retained_users,
    first_value(count(DISTINCT user_id)) OVER (PARTITION BY first_day ORDER BY activity_day) as first_day
_users
FROM (
    SELECT
        user_id,
        toDate(timestamp) as activity_day,
        min(toDate(timestamp)) OVER (PARTITION BY user_id) as first_day
    FROM events
)
GROUP BY first_day, activity_day
)
WHERE retention_day <= 7
ORDER BY first_day, retention_day;

```

7.4 JOINS in ClickHouse

Standard JOIN:

```

SELECT
    e.event_type,
    u.name,
    count() as event_count
FROM events e

```

```
INNER JOIN users u ON e.user_id = u.id
GROUP BY e.event_type, u.name;
```

Dictionary Lookup (Faster than JOIN):

```
-- Create a dictionary
CREATE DICTIONARY user_dict (
    id UInt64,
    name String,
    email String
)
PRIMARY KEY id
SOURCE(CLICKHOUSE(TABLE 'users'))
LAYOUT(HASHED())
LIFETIME(MIN 300 MAX 360);

-- Use dictionary in query
SELECT
    event_type,
    dictGet('user_dict', 'name', user_id) as user_name,
    count() as events
FROM events
GROUP BY event_type, user_name;
```

8. Real-World Use Cases

8.1 Web and Application Analytics

ClickHouse excels at storing and analyzing user behavior data:

Page Views and Sessions:

- Track billions of page views per day
- Real-time session analysis
- User journey mapping
- A/B test result analysis

Example: Yandex.Metrica

- 13+ trillion rows stored
- 20+ billion events per day
- Sub-second dashboard queries
- 400+ ClickHouse servers

8.2 Log Analysis and Observability

Modern observability platforms use ClickHouse for:

Application Logs:

- Centralized log storage

- Full-text search with tokenization
- Log pattern analysis
- Anomaly detection

Infrastructure Metrics:

- Server metrics aggregation
- Container/Kubernetes monitoring
- Network traffic analysis

Distributed Tracing:

- Trace storage and querying
- Span analysis
- Service dependency mapping

8.3 Time Series and IoT

ClickHouse handles time series data efficiently:

Sensor Data:

- Industrial IoT sensors
- Environmental monitoring
- Smart city infrastructure
- Vehicle telemetry

Features for Time Series:

- Efficient time-based partitioning
- Downsampling with TTL
- Built-in time series functions
- Grafana integration

8.4 Business Intelligence

ClickHouse powers BI dashboards with:

Real-Time Dashboards:

- Sub-second query response
- Complex aggregations
- Drill-down capabilities
- Multi-dimensional analysis

Self-Service Analytics:

- Direct SQL access
- Integration with BI tools (Tableau, Superset, Metabase)
- Ad-hoc query support

8.5 Ad-Tech and Click Tracking

The advertising industry heavily uses ClickHouse for:

Impression and Click Tracking:

- Billions of events per hour
- Real-time campaign analytics
- Attribution modeling
- Fraud detection

Real-Time Bidding:

- Bid request logging
- Auction analysis
- Performance optimization

8.6 Financial Analytics

Financial services leverage ClickHouse for:

Market Data Analysis:

- Tick data storage
- Historical price analysis
- Trading pattern detection

Risk Analytics:

- Real-time risk calculations
 - Portfolio analysis
 - Compliance reporting
-

9. When NOT to Use ClickHouse

9.1 High-Frequency Single-Row Updates

ClickHouse's append-only architecture makes it unsuitable for workloads requiring frequent updates to individual rows:

Why It's a Problem:

- Updates are implemented as mutations (rewriting entire parts)
- No row-level locking
- UPDATE/DELETE operations are expensive
- Not designed for OLTP patterns

Better Alternatives: PostgreSQL, MySQL, MongoDB

9.2 Transactional Workloads (ACID Requirements)

ClickHouse does not provide full ACID transaction support:

Limitations:

- No multi-statement transactions
- Eventual consistency for replication
- No rollback capability
- No foreign key constraints

Better Alternatives: PostgreSQL, MySQL, Oracle, SQL Server

9.3 Small Datasets (< Millions of Rows)

ClickHouse's overhead isn't justified for small datasets:

Why It Matters:

- Minimum part size overhead
- Background merge processes
- Complex configuration
- Operational complexity

Better Alternatives: SQLite, PostgreSQL, DuckDB (for embedded analytics)

9.4 Complex JOIN Operations

While ClickHouse supports JOINs, it's not optimized for:

Problematic Patterns:

- Many-to-many JOINs between large tables
- Complex multi-way JOINs
- JOINs without proper distribution keys
- Correlated subqueries

Mitigation Strategies:

- Denormalize data

- Use dictionaries for lookups
- Pre-join in materialized views
- Distribute tables appropriately

Better Alternatives: Traditional data warehouses, Spark SQL

9.5 Primary Key Lookups

Fetching individual rows by ID is inefficient:

Why:

- Columnar storage optimized for scans
- Sparse index requires range scans
- No row-level caching

Better Alternatives: Redis, PostgreSQL, key-value stores

9.6 Summary: ClickHouse Anti-Patterns

Pattern	Problem	Alternative
Frequent UPDATES	Expensive mutations	PostgreSQL
ACID transactions	Not supported	PostgreSQL, MySQL
Small data (<1M rows)	Overhead not justified	SQLite, DuckDB
Point lookups	Inefficient scans	Redis, PostgreSQL

Pattern	Problem	Alternative
Complex JOINS	Memory-intensive	Spark, traditional DW
Blob storage	Not optimized	S3, object storage

10. Production Deployments and Case Studies

10.1 Uber

Use Case: Real-time analytics and observability platform

Scale:

- Trillions of events per day
- Petabytes of data
- Thousands of queries per second

Architecture:

- Multi-cluster deployment
- Custom ingestion pipelines
- Integration with internal tools

10.2 Cloudflare

Use Case: Log analysis and internal analytics platform

Scale:

- Analyzing logs from billions of HTTP requests
- Global CDN observability
- DDoS attack analysis

Benefits:

- Replaced multiple legacy systems
- Dramatic cost reduction
- Faster query performance

10.3 Microsoft

Use Case: Microsoft Titan - data analytics platform

Applications:

- Internal business intelligence
- Product telemetry analysis
- Customer analytics

10.4 Cisco

Use Case: Real-time analytics and data warehousing

Implementation:

- Security analytics
- Network telemetry
- Business intelligence

10.5 Tesla

Use Case: Real-time vehicle analytics and AI/ML

Applications:

- Vehicle telemetry analysis
- Autopilot data processing
- Manufacturing analytics

10.6 Other Notable Users

- **eBay:** Click analytics and search optimization
- **Spotify:** Music streaming analytics
- **Lyft:** Ride-sharing analytics
- **Comcast:** Media and entertainment analytics
- **Bloomberg:** Financial data analysis
- **GitLab:** Product analytics
- **Segment:** Customer data platform

11. Installation, Setup, and Hosting Options

11.1 Self-Hosted Deployment

Docker Quick Start:

```
# Start ClickHouse server
docker run -d --name clickhouse \
  -p 8123:8123 -p 9000:9000 \
  -v clickhouse-data:/var/lib/clickhouse \
  clickhouse/clickhouse-server

# Connect with client
docker exec -it clickhouse clickhouse-client
```

Docker Compose for Persistence:

```
version: '3.8'
services:
  clickhouse:
    image: clickhouse/clickhouse-server
    ports:
      - "8123:8123"
      - "9000:9000"
    volumes:
      - ./data:/var/lib/clickhouse
      - ./config:/etc/clickhouse-server/config.d
```

```
ulimits:  
  nofile:  
    soft: 262144  
    hard: 262144
```

Native Installation (Ubuntu/Debian):

```
# Add repository  
sudo apt-get install -y apt-transport-https ca-certificates dirmngr  
sudo apt-key adv --keyserver hkps://keyserver.ubuntu.com:80 --recv 8919F6BD2B48D754  
  
echo "deb https://packages.clickhouse.com/deb stable main" | sudo tee \  
  /etc/apt/sources.list.d/clickhouse.list  
  
# Install  
sudo apt-get update  
sudo apt-get install -y clickhouse-server clickhouse-client  
  
# Start service  
sudo systemctl start clickhouse-server  
sudo systemctl enable clickhouse-server  
  
# Connect  
clickhouse-client
```

Kubernetes Deployment:

ClickHouse can be deployed on Kubernetes using:

- Official Helm charts
- ClickHouse Operator (recommended for production)
- Custom manifests

11.2 Cloud Hosting Options

ClickHouse Cloud (Official Managed Service):

- Fully managed by ClickHouse Inc.
- Automatic scaling
- Built-in monitoring
- Pay-per-use pricing
- Multiple cloud providers (AWS, GCP, Azure)

Altinity.Cloud:

- Managed ClickHouse by Altinity
- Kubernetes-native deployment
- Enterprise support
- Available on AWS and GCP

Aiven for ClickHouse:

- Part of Aiven's database platform
- Multi-cloud support
- Integrated with other Aiven services

Cloud Provider Marketplaces:

- AWS Marketplace
- GCP Marketplace
- Azure Marketplace

11.3 Client Connectivity

Native CLI Client:

```
clickhouse-client --host localhost --port 9000 --user default
```

HTTP API:

```
# Query via HTTP
curl 'http://localhost:8123/?query=SELECT%201'

# With authentication
curl 'http://localhost:8123/' \
  --user 'default:password' \
  --data 'SELECT * FROM system.tables'
```


Programming Language Drivers:

Language	Driver
Python	clickhouse-driver, clickhouse-connect
Go	clickhouse-go
Java	clickhouse-jdbc
Node.js	clickhouse-js
Rust	clickhouse-rs
PHP	smi2/phpclickhouse
.NET	ClickHouse.Client

11.4 Scaling Strategies

Vertical Scaling:

- Add CPU cores for faster queries
- Add RAM for larger working sets
- Use faster NVMe storage for I/O-bound workloads
- Single-node can handle TB-scale data

Horizontal Scaling:

- Add shards for more storage capacity
- Add replicas for read scaling and HA

- Use Distributed tables for transparent access
 - ClickHouse Keeper for coordination
-

12. Best Practices and Optimization

12.1 Schema Design

Choose the Right Data Types:

```
-- Bad: Using larger types than necessary
user_id String,  -- 8+ bytes per value
count Int64      -- 8 bytes, overkill for small counts

-- Good: Minimal types
user_id UInt64,   -- Fixed 8 bytes, efficient operations
count UInt16     -- 2 bytes, sufficient for 0-65535
```

Use LowCardinality for Low-Cardinality Strings:

```
-- Bad: Regular String for repeated values
status String,  -- 'pending', 'completed', 'failed'

-- Good: LowCardinality (dictionary encoding)
status LowCardinality(String)  -- 1-2 bytes vs 10+ bytes
```

Design Effective ORDER BY Clauses:

1. Put frequently filtered columns first
2. Order by cardinality (low to high)
3. Include timestamp for time-range queries

```
-- Good ORDER BY for event data
ORDER BY (event_type, toDate(timestamp), user_id)
```

12.2 Query Optimization

Filter Early, Select Late:

```
-- Bad: Process all data, filter later
SELECT user_id, sum(amount)
FROM events
GROUP BY user_id
HAVING sum(amount) > 100

-- Better: Filter early with WHERE
SELECT user_id, sum(amount)
FROM events
WHERE amount > 0 -- Reduces data scanned
GROUP BY user_id
HAVING sum(amount) > 100
```

Use PREWHERE for Large Tables:

```
-- PREWHERE filters before reading all columns
SELECT * FROM events
PREWHERE event_type = 'purchase'
WHERE user_id > 1000
```

****Avoid SELECT ***:**

```
-- Bad: Reads all columns
SELECT * FROM events LIMIT 100

-- Good: Only requested columns
SELECT timestamp, user_id, event_type FROM events LIMIT 100
```

12.3 Insert Optimization

Batch Inserts:

```
-- Bad: Individual inserts
INSERT INTO events VALUES (...); -- Repeated 1000x

-- Good: Batch insert
INSERT INTO events VALUES
    (...),
```

```
(...),  
-- ... 1000+ rows  
(...);
```

Use Asynchronous Inserts for High-Frequency Streams:

```
SET async_insert = 1;  
SET wait_for_async_insert = 0;  
-- Inserts are batched automatically
```

Consider Buffer Tables:

```
CREATE TABLE events_buffer AS events  
ENGINE = Buffer(default, events, 16, 10, 100, 10000, 100000, 1000000, 10000000);  
  
-- Write to buffer, automatic flush to main table  
INSERT INTO events_buffer VALUES (...);
```

12.4 Monitoring and Maintenance

System Tables for Monitoring:

```
-- Query performance  
SELECT * FROM system.query_log ORDER BY event_time DESC LIMIT 10;
```

```
-- Table sizes
SELECT table, sum(bytes) as size, sum(rows) as rows
FROM system.parts
GROUP BY table
ORDER BY size DESC;

-- Merge activity
SELECT * FROM system.merges;

-- Replication status
SELECT * FROM system.replication_queue;
```

Regular Maintenance:

```
-- Force merge of small parts
OPTIMIZE TABLE events FINAL;

-- Check table health
CHECK TABLE events;

-- Analyze table statistics
ANALYZE TABLE events;
```

13. Comparison with Other OLAP Systems

13.1 ClickHouse vs. PostgreSQL

Aspect	ClickHouse	PostgreSQL
Storage Model	Columnar	Row-oriented
Best For	Analytics	Transactions + Analytics
Aggregation Speed	10-100x faster	Baseline
Update/Delete	Expensive	Efficient
ACID Support	Limited	Full
Compression	10-20x	2-4x

When to Choose ClickHouse: Pure analytics, large datasets, real-time dashboards

When to Choose PostgreSQL: Mixed workloads, ACID requirements, smaller datasets

13.2 ClickHouse vs. Apache Druid

Aspect	ClickHouse	Druid
Architecture	Simpler	More complex (Coordinator, Broker, Historical, etc.)
Query Language	SQL	SQL + Druid-specific
Real-Time Ingestion	Native	Via Kafka ingestion
Pre-Aggregation	Materialized Views	Built-in roll-up
Operational Complexity	Lower	Higher

13.3 ClickHouse vs. Apache Spark

Aspect	ClickHouse	Spark
Query Latency	Milliseconds	Seconds to minutes
Storage	Built-in	External (Parquet, Delta, etc.)
Best For	Interactive queries	Batch processing, ML
Concurrency	High	Lower
Resource Usage	Efficient	Higher memory requirements

13.4 ClickHouse vs. Snowflake/BigQuery

Aspect	ClickHouse	Cloud DW (Snowflake/BQ)
Deployment	Self-hosted or Cloud	Cloud-only
Cost Model	Infrastructure cost	Per-query pricing
Performance	Often faster	Good, scalable
Vendor Lock-in	None	Significant
Operational Burden	Higher	None

14. Resources and Further Reading

14.1 Official Documentation

- **ClickHouse Documentation:** <https://clickhouse.com/docs>
- **Academic Overview:** https://clickhouse.com/docs/academic_overview
- **SQL Reference:** <https://clickhouse.com/docs/sql-reference>

14.2 Performance and Benchmarks

- **ClickBench:** <https://clickhouse.com/benchmark>
- **JSON Benchmark Blog Post:** <https://clickhouse.com/blog/json-bench-clickhouse-vs-mongodb-elasticsearch-duckdb-postgresql>

14.3 Architecture and Internals

- **MergeTree Engine Documentation:** <https://clickhouse.com/docs/engines/table-engines/mergetree-family/mergetree>
- **Cluster Deployment Guide:** <https://clickhouse.com/docs/architecture/cluster-deployment>

14.4 Community Resources

- **ClickHouse GitHub:** <https://github.com/ClickHouse/ClickHouse>
- **ClickHouse Blog:** <https://clickhouse.com/blog>
- **Community Slack:** <https://clickhouse.com/slack>

14.5 Third-Party Guides

- **Altinity Knowledge Base:** <https://kb.altinity.com>
- **Medium Article on ClickHouse Architecture:** <https://medium.com/@imanjafari71/a-practical-guide-to-clickhouse-pros-cons-architecture-installation-clusteringand-best-91477df8eb>

14.6 Tools and Integrations

- **Grafana ClickHouse Plugin:** <https://grafana.com/grafana/plugins/grafana-clickhouse-datasource/>
 - **Apache Superset:** <https://superset.apache.org>
 - **Metabase:** <https://www.metabase.com>
 - **dbt-clickhouse:** <https://github.com/ClickHouse/dbt-clickhouse>
-

Conclusion

ClickHouse represents a paradigm shift in analytical database technology. By combining columnar storage, aggressive compression, vectorized execution, and real-time capabilities, it enables organizations to process and analyze data at scales and speeds that were previously impossible or prohibitively expensive.

Key Takeaways:

1. **Columnar storage** enables 10-20x compression and dramatically faster analytics
2. **Vectorized execution** with SIMD instructions achieves billions of rows per second
3. **MergeTree engine family** provides flexibility for various use cases
4. **Zero external dependencies** simplify deployment and operations
5. **Real-time ingestion** makes data immediately queryable
6. **Linear scalability** handles growing data volumes efficiently

Best Use Cases: Analytics, logs, time series, BI dashboards, real-time reporting

Avoid For: Transactional workloads, frequent updates, small datasets, complex JOINS

Whether powering a startup's analytics dashboard or processing billions of events for a Fortune 500 company, ClickHouse provides the performance and flexibility needed for modern data-intensive applications.

Document generated based on the ClickHouse presentation by Julius Ball and Anton Pfurtscheller, expanded with comprehensive technical details and best practices.